

data struct PLUS

panyanppyy

- 由于篇幅限制，某些地方不会详细展开，如果需要更详细地了解相关内容，可以前往[OI-Wiki](#)，同时，文中引用的图片大多来源于此。

前言

- 由于篇幅限制，某些地方不会详细展开，如果需要更详细地了解相关内容，可以前往[OI-Wiki](#)，同时，文中引用的图片大多来源于此。
- 讲课的时候我把一些比较难的例题放到了讲课中，不会要及时说出来。

前言

- 由于篇幅限制，某些地方不会详细展开，如果需要更详细地了解相关内容，可以前往[OI-Wiki](#)，同时，文中引用的图片大多来源于此。
- 讲课的时候我把一些比较难的例题放到了讲课中，不会要及时说出来。
- 带 * 的是选学内容，但是尽量多学一点也是好的。

前言

- 由于篇幅限制，某些地方不会详细展开，如果需要更详细地了解相关内容，可以前往[OI-Wiki](#)，同时，文中引用的图片大多来源于此。
- 讲课的时候我把一些比较难的例题放到了讲课中，不会要及时说出来。
- 带 * 的是选学内容，但是尽量多学一点也是好的。
- 有问题可以 QQ(910256688) 联系，但是不经常在。

前言

- 由于篇幅限制，某些地方不会详细展开，如果需要更详细地了解相关内容，可以前往[OI-Wiki](#)，同时，文中引用的图片大多来源于此。
- 讲课的时候我把一些比较难的例题放到了讲课中，不会要及时说出来。
- 带 * 的是选学内容，但是尽量多学一点也是好的。
- 有问题可以 QQ(910256688) 联系，但是不经常在。
- 文章中代码片段比较多，给基础比较差的同学理解，可以跳过。

前言

- 由于篇幅限制，某些地方不会详细展开，如果需要更详细地了解相关内容，可以前往[OI-Wiki](#)，同时，文中引用的图片大多来源于此。
- 讲课的时候我把一些比较难的例题放到了讲课中，不会要及时说出来。
- 带 * 的是选学内容，但是尽量多学一点也是好的。
- 有问题可以 QQ(910256688) 联系，但是不经常在。
- 文章中代码片段比较多，给基础比较差的同学理解，可以跳过。
- 例题的 Title 都有超链接。

前言

- 由于篇幅限制，某些地方不会详细展开，如果需要更详细地了解相关内容，可以前往[OI-Wiki](#)，同时，文中引用的图片大多来源于此。
- 讲课的时候我把一些比较难的例题放到了讲课中，不会要及时说出来。
- 带 * 的是选学内容，但是尽量多学一点也是好的。
- 有问题可以 QQ(910256688) 联系，但是不经常在。
- 文章中代码片段比较多，给基础比较差的同学理解，可以跳过。
- 例题的 Title 都有超链接。
- 内容很多，作业很多，你们忍一下。

目录

1 ST 表

- 倍增
- ST table
- 例题

2 树状数组

3 线段树

4 珂朵莉树

5 可持久化线段树 (主席树)

6 我们的红太阳

什么是倍增

- 因为 ST 表要用，所以讲讲。

什么是倍增

- 因为 ST 表要用，所以讲讲。
- 其实倍增本质上就是一个 dp。

- 数轴上有 n 个点 a_i , 有一只青蛙在这些点上跳。
规则是每次向距当前点第 k 小的点跳, 如果有相同距离则向下标较小的跳。
求从每个点出发跳了 m 次后在哪里。
- $1 \leq n, k \leq 10^6, 1 \leq m \leq 10^{18}, 1 \leq a_i \leq 10^{18}$

- 第 k 小的点可以用滑动窗口直接求出来。

- 第 k 小的点可以用滑动窗口直接求出来。
- 设 $f_{i,j}$ 表示点 i 跳 j 次到的点。

- 第 k 小的点可以用滑动窗口直接求出来。
- 设 $f_{i,j}$ 表示点 i 跳 j 次到的点。
- 转移: $f_{i,j} = f_{f_{i,1},j-1}$

- 第 k 小的点可以用滑动窗口直接求出来。
- 设 $f_{i,j}$ 表示点 i 跳 j 次到的点。
- 转移: $f_{i,j} = f_{f_{i,1},j-1}$
- 单次询问的时间复杂度是 $\mathcal{O}(1)$ 。

- 第 k 小的点可以用滑动窗口直接求出来。
- 设 $f_{i,j}$ 表示点 i 跳 j 次到的点。
- 转移: $f_{i,j} = f_{f_{i,1},j-1}$
- 单次询问的时间复杂度是 $\mathcal{O}(1)$ 。
- 但是空间非常大, 是 $\mathcal{O}(nm)$ 。

- 第 k 小的点可以用滑动窗口直接求出来。
- 设 $f_{i,j}$ 表示点 i 跳 j 次到的点。
- 转移: $f_{i,j} = f_{f_{i,1},j-1}$
- 单次询问的时间复杂度是 $\mathcal{O}(1)$ 。
- 但是空间非常大, 是 $\mathcal{O}(nm)$ 。
- 我们需要一种空间 $\mathcal{O}(n \log m)$, 时间 $\mathcal{O}(n \log m)$ 的 dp。

- 考虑优化这个 dp。

- 考虑优化这个 dp。
- 其中有很多状态都是没有用的，比如 $f_{i,3}$ ，可以通过 $f_{f_{i,1},2}$ 求出。

- 考虑优化这个 dp。
- 其中有很多状态都是没有用的，比如 $f_{i,3}$ ，可以通过 $f_{f_{i,1},2}$ 求出。
- 所以考虑二进制分解，用 $f_{i,j}$ 表示点 i 跳 2^j 步到的点。

- 考虑优化这个 dp。
- 其中有很多状态都是没有用的，比如 $f_{i,3}$ ，可以通过 $f_{f_{i,1},2}$ 求出。
- 所以考虑二进制分解，用 $f_{i,j}$ 表示点 i 跳 2^j 步到的点。
- 转移： $f_{i,j} = f_{f_{i,j-1},j-1}$ 。

- 考虑优化这个 dp。
- 其中有很多状态都是没有用的，比如 $f_{i,3}$ ，可以通过 $f_{f_{i,1},2}$ 求出。
- 所以考虑二进制分解，用 $f_{i,j}$ 表示点 i 跳 2^j 步到的点。
- 转移： $f_{i,j} = f_{f_{i,j-1},j-1}$ 。
- 这样就可以在 $\mathcal{O}(n \log m)$ 的时间内求出所有点了。

- 考虑优化这个 dp。
- 其中有很多状态都是没有用的，比如 $f_{i,3}$ ，可以通过 $f_{f_{i,1},2}$ 求出。
- 所以考虑二进制分解，用 $f_{i,j}$ 表示点 i 跳 2^j 步到的点。
- 转移： $f_{i,j} = f_{f_{i,j-1},j-1}$ 。
- 这样就可以在 $O(n \log m)$ 的时间内求出所有点了。
- 空间滚动一下就变成 $O(n)$ 了。

什么是 ST 表

- ST 表是用于解决**可重复贡献问题** 的数据结构。

什么是 ST 表

- ST 表是用于解决**可重复贡献问题** 的数据结构。
- 因为 ST 表在询问的时候是一段前缀和一段后缀贡献拼起来，中间有一段重叠部分。

什么是 ST 表

- ST 表是用于解决**可重复贡献问题** 的数据结构。
- 因为 ST 表在询问的时候是一段前缀和一段后缀贡献拼起来，中间有一段重叠部分。
- 比如 RMQ 和区间 gcd 就是典型的可重复贡献问题。

- 给定一个长度为 n 的序列 a , q 次询问, 每次询问求区间 $[l, r]$ 中的最小值。

- 给定一个长度为 n 的序列 a , q 次询问, 每次询问求区间 $[l, r]$ 中的最小值。
- $1 \leq n \leq 10^5, 1 \leq q \leq 2 \times 10^7$

- 设 $f_{i,j}$ 表示从 i 开始, 向后 2^j 个数内的最小值。

ST 表

- 设 $f_{i,j}$ 表示从 i 开始, 向后 2^j 个数内的最小值。
- 转移: $f_{i,j} = \min(f_{i,j-1}, f_{f_{i,j-1},j-1})$ 。

ST 表

- 设 $f_{i,j}$ 表示从 i 开始, 向后 2^j 个数内的最小值。
- 转移: $f_{i,j} = \min(f_{i,j-1}, f_{i+j-1,j-1})$ 。
- 询问:

```
int query(int l,int r){  
    int k=__lg(r-l+1);  
    return min(f[l][k],f[r-(1<<k)+1][k]);  
}
```


ST 表

- 设 $f_{i,j}$ 表示从 i 开始, 向后 2^j 个数内的最小值。
- 转移: $f_{i,j} = \min(f_{i,j-1}, f_{f_{i,j-1},j-1})$ 。
- 询问:

```
int query(int l,int r){
    int k=__lg(r-l+1);
    return min(f[l][k],f[r-(1<<k)+1][k]);
}
```

- `__lg` 可以快速求出 $\log_2(n)$, **但是** `__lg(0)` 会死得很惨, 死亡名单: 蒟酱。

- 注意：ST 表求区间 gcd 预处理时间复杂度不是 $\mathcal{O}(n \log n \log w)$ 考虑每次 gcd 都会让值减半，最多减 $\log w$ 次，加上循环的 $\log n$ ，就是 $\mathcal{O}(n(\log n + \log w))$ 。

区间 gcd

- 注意：ST 表求区间 gcd 预处理时间复杂度不是 $\mathcal{O}(n \log n \log w)$ 考虑每次 gcd 都会让值减半，最多减 $\log w$ 次，加上循环的 $\log n$ ，就是 $\mathcal{O}(n(\log n + \log w))$ 。
- 单次询问时间复杂度还是 $\mathcal{O}(\log w)$ 。

区间 gcd

- 注意：ST 表求区间 gcd 预处理时间复杂度不是 $\mathcal{O}(n \log n \log w)$ 考虑每次 gcd 都会让值减半，最多减 $\log w$ 次，加上循环的 $\log n$ ，就是 $\mathcal{O}(n(\log n + \log w))$ 。
- 单次询问时间复杂度还是 $\mathcal{O}(\log w)$ 。
- 空间复杂度还是 $\mathcal{O}(n \log n)$

01 序列

- 给你一个长度为 n 的 01 序列 a , m 次询问, 支持 2 种询问:
- 1 $l\ r$ 表示询问 l 到 r 区间的最长不下降子序列的长度。
2 $l\ r$ 表示询问 l 到 r 区间的最长上升子序列的长度。
- $1 \leq n \leq 10^6, 1 \leq m \leq 5 \times 10^6, a_i \in \{0, 1\}$ 1.2s

01 序列

- 询问 2 的答案就是问有无形如 0,1 的子序列。

01 序列

- 询问 2 的答案就是问有无形如 0, 1 的子序列。
- 可以发现询问 1 的答案就是 $\max_{i=l-1}^r pre_{0,i} - pre_{0,l-1} + pre_{1,r} - pre_{1,i}$

01 序列

- 询问 2 的答案就是问有无形如 0, 1 的子序列。
- 可以发现询问 1 的答案就是 $\max_{i=l-1}^r pre_{0,i} - pre_{0,l-1} + pre_{1,r} - pre_{1,i}$
- 考虑这个东西怎么优化。

01 序列

- 询问 2 的答案就是问有无形如 0, 1 的子序列。
- 可以发现询问 1 的答案就是 $\max_{i=l-1}^r pre_{0,i} - pre_{0,l-1} + pre_{1,r} - pre_{1,i}$
- 考虑这个东西怎么优化。
- 转化式子: $\max_{i=l-1}^r (pre_{0,i} + suf_{1,i}) - pre_{0,l-1} - suf_{1,r+1}$

01 序列

- 询问 2 的答案就是问有无形如 0, 1 的子序列。
- 可以发现询问 1 的答案就是 $\max_{i=l-1}^r pre_{0,i} - pre_{0,l-1} + pre_{1,r} - pre_{1,i}$
- 考虑这个东西怎么优化。
- 转化式子: $\max_{i=l-1}^r (pre_{0,i} + suf_{1,i}) - pre_{0,l-1} - suf_{1,r+1}$
- 然后这个东西就可以用 ST 表优化到 $\mathcal{O}(1)$ 了。

目录

1 ST 表

2 树状数组

- 区间和
- lowbit
- fenwick
- 树状数组上二分
- 例题

3 线段树

4 珂朵莉树

5 可持久化线段树 (主席树)

6 我们的红太阳

怎么求区间和

- 如何快速求出 $[l, r]$ 区间和呢?

怎么求区间和

- 如何快速求出 $[l, r]$ 区间和呢?
- 假如你会前缀和, $s_i = \sum_{j=1}^i a_j$

怎么求区间和

- 如何快速求出 $[l, r]$ 区间和呢?
- 假如你会前缀和, $s_i = \sum_{j=1}^i a_j$
- 你会发现, $S(l, r) = s_r - s_{l-1}$, 减一是为了包括 a_l 的值。

什么是 lowbit

- lowbit 就是一个数二进制下的最后一位。

什么是 lowbit

- lowbit 就是一个数二进制下的最后一位。
- 比如：
8 $(1000)_2$ 的 lowbit 就是 8 $(1000)_2$
6 $(110)_2$ 的 lowbit 就是 2 $(10)_2$

怎么求 lowbit

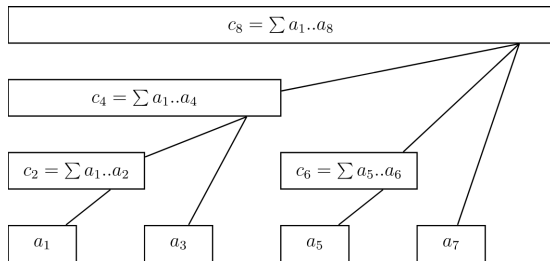
- 可以通过 $-x \& x$ 快速求出。

怎么求 lowbit

- 可以通过 $-x \& x$ 快速求出。
- 原理： $-x = \neg x + 1$ 相反数相当于取反再加一，取反可以把所有 0 变成 1 然后从后往前进位就可以找到原数的最后一个 1，然后和原数取与就可以去除多余的数，获得 lowbit 了。

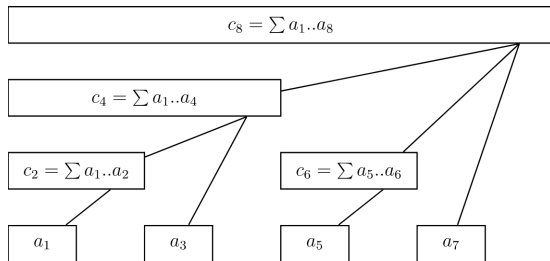
什么是树状数组

- 下面这张图展示了树状数组的工作原理：



什么是树状数组

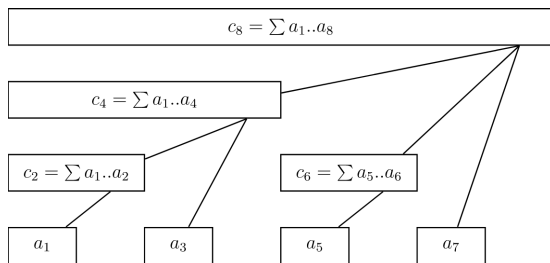
- 下面这张图展示了树状数组的工作原理：



- 可以发现, c_i 可以表示成从 $i - \text{lowbit}(i) + 1$ 到 i 的区间信息。

什么是树状数组

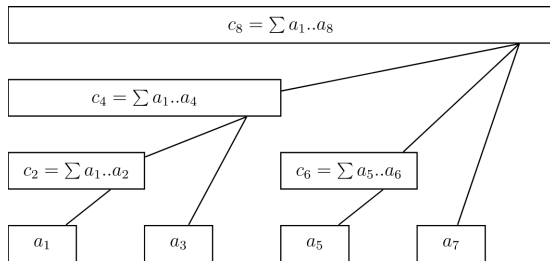
- 下面这张图展示了树状数组的工作原理：



- 可以发现, c_i 可以表示成从 $i - \text{lowbit}(i) + 1$ 到 i 的区间信息。
- 就可以对一个数二进制分解来获得一个前缀的信息。

什么是树状数组

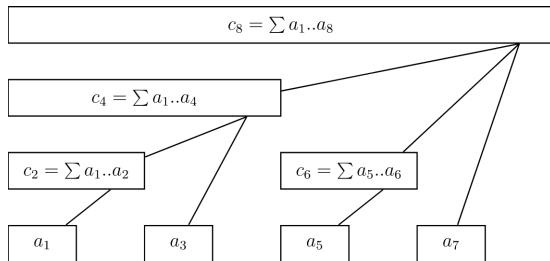
- 下面这张图展示了树状数组的工作原理：



- 可以发现， c_i 可以表示成从 $i - \text{lowbit}(i) + 1$ 到 i 的区间信息。
- 就可以对一个数二进制分解来获得一个前缀的信息。
- 对于每次询问都要从高位到低位求和。

什么是树状数组

- 下面这张图展示了树状数组的工作原理：



- 可以发现， c_i 可以表示成从 $i - \text{lowbit}(i) + 1$ 到 i 的区间信息。
- 就可以对一个数二进制分解来获得一个前缀的信息。
- 对于每次询问都要从高位到低位求和。
- 对于修改其中一个数的值都要修改所有包含它的区间。

怎么用树状数组

- 树状数组可以很优秀地解决单点修改，区间求和的问题。

怎么用树状数组

- 树状数组可以很优秀地解决单点修改，区间求和的问题。
- 结合差分就可以通过 $\text{query}(r) - \text{query}(l - 1)$ 来获得区间和了。

怎么用树状数组

- 树状数组可以很优秀地解决单点修改，区间求和的问题。
- 结合差分就可以通过 $\text{query}(r) - \text{query}(l - 1)$ 来获得区间和了。
- 注意下标千万不能从 0 开始，否则取 lowbit 一直是 0，就死循环了。

怎么用树状数组

- 树状数组可以很优秀地解决单点修改，区间求和的问题。
- 结合差分就可以通过 $\text{query}(r) - \text{query}(l - 1)$ 来获得区间和了。
- 注意下标千万不能从 0 开始，否则取 lowbit 一直是 0，就死循环了。
- 有人让我说树状数组来源，那我就说下：是线段树去掉一些无用节点的结果。

代码实现

- 实现如下:

```
int low(int x){return -x&x;}
void add(int x,int k){
    for(;x<=n;x+=low(x))t[x]+=k;
}
int query(int x){
    int res=0;
    for(;x;x^=low(x))res+=t[x];
    return res;
}
int query(int l,int r){return query(r)-query(l-1);}
```

怎么用树状数组

- 同样的，树状数组可以很优秀地解决区间修改，单点查询。

怎么用树状数组

- 同样的，树状数组可以很优秀地解决区间修改，单点查询。
- 可以令 $t_l \leftarrow t_l + x, t_{r+1} \leftarrow t_{r+1} - x$ 来进行区间修改。

怎么用树状数组

- 同样的，树状数组可以很优秀地解决区间修改，单点查询。
- 可以令 $t_l \leftarrow t_l + x, t_{r+1} \leftarrow t_{r+1} - x$ 来进行区间修改。
- 然后直接单点查询即可。

怎么用树状数组

- 解决区间修改，区间查询会比较繁琐。

怎么用树状数组

- 解决区间修改，区间查询会比较繁琐。
- 对于差分数组 b 可以推导：

怎么用树状数组

- 解决区间修改，区间查询会比较繁琐。
- 对于差分数组 b 可以推导：
-

$$\begin{aligned}\sum_{i=1}^r a_i &= \sum_{i=1}^r \sum_{j=1}^i b_j \\ &= \sum_{i=1}^r b_i \times (r - i + 1) \\ &= \sum_{i=1}^r b_i \times (r + 1) - \sum_{i=1}^r b_i \times i\end{aligned}$$

怎么用树状数组

- 解决区间修改，区间查询会比较繁琐。
- 对于差分数组 b 可以推导：
-

$$\begin{aligned}\sum_{i=1}^r a_i &= \sum_{i=1}^r \sum_{j=1}^i b_j \\ &= \sum_{i=1}^r b_i \times (r - i + 1) \\ &= \sum_{i=1}^r b_i \times (r + 1) - \sum_{i=1}^r b_i \times i\end{aligned}$$

- 然后就能分成两个数组维护了。

怎么用树状数组

- 除了求和，树状数组还能维护单调增的 $\max$ 。

怎么用树状数组

- 除了求和，树状数组还能维护单调增的 $\max$ 。
- 如果不保证单调性有一种 $\mathcal{O}(\log^2 n)$ 的方法，但是没什么用，就不讲了。

【模板】普通平衡树

- 维护一个集合, 6 个操作:
 1. 插入 x
 2. 删除 x
 3. 求 x 的排名
 4. 求排名为 x 的数
 5. 求 x 的前驱
 6. 求 x 的后继
- $1 \leq q \leq 10^6$

树状数组上二分

- 直接对值域建一颗树状数组，统计出现次数，类似于一个桶，也就是权值树状数组。

树状数组上二分

- 直接对值域建一颗树状数组，统计出现次数，类似于一个桶，也就是权值树状数组。
- 插入 x : $t_x \leftarrow t_x + 1$

树状数组上二分

- 直接对值域建一颗树状数组，统计出现次数，类似于一个桶，也就是权值树状数组。
- 插入 x : $t_x \leftarrow t_x + 1$
- 删除 x : $t_x \leftarrow t_x - 1$

树状数组上二分

- 直接对值域建一颗树状数组，统计出现次数，类似于一个桶，也就是权值树状数组。
- 插入 x : $t_x \leftarrow t_x + 1$
- 删除 x : $t_x \leftarrow t_x - 1$
- 求 x 的排名: $(\sum_{i=1}^{x-1} t_i) + 1$

树状数组上二分

- 求排名为 x 的数：考虑要找到一个最大的 k 满足 $(\sum_{i=1}^{k-1} t_i) + 1 \leq x$ 。
看上去就很能二分的样子，因为在树状数组上，所以考虑像倍增一样跳就行了。

```
int kth(int k){
    int r=0,s=0;
    for(int i=20;~i;i--){
        if(r+(1<<i)>m) continue;
        if(s+t[r+(1<<i)]<k) r+=1<<i,s+=t[r];
    }
    return b[r];
}
```

树状数组上二分

- 求排名为 x 的数：考虑要找到一个最大的 k 满足 $(\sum_{i=1}^{k-1} t_i) + 1 \leq x$ 。
看上去就很能二分的样子，因为在树状数组上，所以考虑像倍增一样跳就行了。

```
int kth(int k){
    int r=0,s=0;
    for(int i=20;~i;i--){
        if(r+(1<<i)>m) continue;
        if(s+t[r+(1<<i)]<k) r+=1<<i,s+=t[r];
    }
    return b[r];
}
```

- 求 x 前驱： $\text{kth}(\text{query}(x) - 1)$ 。

树状数组上二分

- 求排名为 x 的数：考虑要找到一个最大的 k 满足 $(\sum_{i=1}^{k-1} t_i) + 1 \leq x$ 。
看上去就很能二分的样子，因为在树状数组上，所以考虑像倍增一样跳就行了。

```
int kth(int k){
    int r=0,s=0;
    for(int i=20;~i;i--){
        if(r+(1<<i)>m) continue;
        if(s+t[r+(1<<i)]<k)r+=(1<<i),s+=t[r];
    }
    return b[r];
}
```

- 求 x 前驱： $\text{kth}(\text{query}(x) - 1)$ 。
- 求 x 后继： $\text{kth}(\text{query}(x + 1))$ 。

树状数组上二分

- 更多关于权值树状数组/树状数组二分的内容可以去[这里](#)自行了解。

逆序对

- 给你一个长度为 n 的序列 a , 求 $i < j, a_i > a_j$ 的对数。
- $1 \leq n \leq 5 \times 10^5, 1 \leq a_i \leq 10^9$

逆序对

- 先离散化。

逆序对

- 先离散化。
- 然后从后往前做题目就变成了已经比当前这个数小的个数。

逆序对

- 先离散化。
- 然后从后往前做题目就变成了已经比当前这个数小的个数。
- 直接用树状数组询问就可以了。

方伯伯的玉米田

- 给你一个长度为 n 的序列 a , 你可以让最多 k 个区间整体 $+1$, 问最后序列的最长不下降子序列的长度。
- $2 \leq n < 10^4, 2 \leq k \leq 500, 1 \leq a_i \leq 5000$ 3s

方伯伯的玉米田

- 显然增加一段后缀更优。

方伯伯的玉米田

- 显然增加一段后缀更优。
- 设 $f_{i,j}$ 表示当前考虑到第 i 个用了 j 次区间加以后的最长不下降子序列长度。

方伯伯的玉米田

- 显然增加一段后缀更优。
- 设 $f_{i,j}$ 表示当前考虑到第 i 个用了 j 次区间加以后的最长不下降子序列长度。
- 转移 $f_{i,j} = \max_{x \leq i, y \leq j, a_x + y \leq a_i + j} (f_{x,y} + 1)$

方伯伯的玉米田

- 显然增加一段后缀更优。
- 设 $f_{i,j}$ 表示当前考虑到第 i 个用了 j 次区间加以后的最长不下降子序列长度。
- 转移 $f_{i,j} = \max_{x \leq i, y \leq j, a_x + y \leq a_i + j} (f_{x,y} + 1)$
- 直接维护一个二维树状数组维护前缀 max 就可以了。

方伯伯的玉米田

- 显然增加一段后缀更优。
- 设 $f_{i,j}$ 表示当前考虑到第 i 个用了 j 次区间加以后的最长不下降子序列长度。
- 转移 $f_{i,j} = \max_{x \leq i, y \leq j, a_x + y \leq a_i + j} (f_{x,y} + 1)$
- 直接维护一个二维树状数组维护前缀 max 就可以了。
- 时间复杂度 $\mathcal{O}(nk \log n \log(w + k))$ 。

目录

1 ST 表

2 树状数组

3 线段树

- 线段树
- 懒标记
- 动态开点
- 线段树二分

- 线段树分治
- 权值线段树的合并和 * 分裂
- 历史最值和 * 区间最值
- 均摊
- 扫描线

4 珂朵莉树

5 可持久化线段树 (主席树)

6 我们的红太阳

什么是线段树

- 线段树是算法竞赛中常用的用来维护 **区间信息** 的数据结构。

什么是线段树

- 线段树是算法竞赛中常用的用来维护 **区间信息** 的数据结构。
- 线段树可以在 $\mathcal{O}(\log n)$ 的时间复杂度内实现单点修改、区间修改、区间查询 (区间求和, 区间最值) 等操作。

什么是线段树

- 线段树是算法竞赛中常用的用来维护 **区间信息** 的数据结构。
- 线段树可以在 $\mathcal{O}(\log n)$ 的时间复杂度内实现单点修改、区间修改、区间查询 (区间求和, 区间最值) 等操作。
- 线段树维护的是满足半群的信息。

什么是线段树

- 线段树是算法竞赛中常用的用来维护 **区间信息** 的数据结构。
- 线段树可以在 $\mathcal{O}(\log n)$ 的时间复杂度内实现单点修改、区间修改、区间查询 (区间求和, 区间最值) 等操作。
- 线段树维护的是满足半群的信息。
- 半群可以简单理解为一个信息集合和一个运算符, 表示为 $(G, \cdot)$ 。

什么是线段树

- 线段树是算法竞赛中常用的用来维护 **区间信息** 的数据结构。
- 线段树可以在 $\mathcal{O}(\log n)$ 的时间复杂度内实现单点修改、区间修改、区间查询 (区间求和, 区间最值) 等操作。
- 线段树维护的是满足半群的信息。
- 半群可以简单理解为一个信息集合和一个运算符, 表示为 $(G, \cdot)$ 。
- 满足以下条件:

什么是线段树

- 线段树是算法竞赛中常用的用来维护 **区间信息** 的数据结构。
- 线段树可以在 $\mathcal{O}(\log n)$ 的时间复杂度内实现单点修改、区间修改、区间查询 (区间求和, 区间最值) 等操作。
- 线段树维护的是满足半群的信息。
- 半群可以简单理解为一个信息集合和一个运算符, 表示为 $(G, \cdot)$ 。
- 满足以下条件:
 1. $\forall x \in S, \forall y \in S, x \cdot y \in S$

什么是线段树

- 线段树是算法竞赛中常用的用来维护 **区间信息** 的数据结构。
- 线段树可以在 $\mathcal{O}(\log n)$ 的时间复杂度内实现单点修改、区间修改、区间查询 (区间求和, 区间最值) 等操作。
- 线段树维护的是满足半群的信息。
- 半群可以简单理解为一个信息集合和一个运算符, 表示为 $(G, \cdot)$ 。
- 满足以下条件:
 1. $\forall x \in S, \forall y \in S, x \cdot y \in S$
 2. $(a \cdot b) \cdot c = a \cdot (b \cdot c)$

什么是线段树

- 下面这张图展示了线段树的工作原理：

d[1]=60 [1,5]			
d[2]=33 [1,3]		d[3]=27 [4,5]	
d[4]=21 [1,2]	d[5]=12 [3,3]	d[6]=13 [4,4]	d[7]=14 [5,5]
d[8]=10 [1,1]	d[9]=11 [2,2]		

什么是线段树

- 下面这张图展示了线段树的工作原理：

d[1]=60 [1,5]			
d[2]=33 [1,3]		d[3]=27 [4,5]	
d[4]=21 [1,2]	d[5]=12 [3,3]	d[6]=13 [4,4]	d[7]=14 [5,5]
d[8]=10 [1,1]	d[9]=11 [2,2]		

- 可以发现，一个节点表示一个区间的信息，从上到每个区间大小不断减半直到变成原序列上的一个点。

什么是线段树

- 每次减半可以保证线段树层数在 $\mathcal{O}(\log n)$ 。

什么是线段树

- 每次减半可以保证线段树层数在 $\mathcal{O}(\log n)$ 。
- 这样每次单点修改和区间询问就可以让时间复杂度稳定在 $\mathcal{O}(\log n)$ 。

什么是线段树

- 每次减半可以保证线段树层数在 $\mathcal{O}(\log n)$ 。
- 这样每次单点修改和区间询问就可以让时间复杂度稳定在 $\mathcal{O}(\log n)$ 。
- 线段树节点数可以证明不超过 $4n$ ，贴个证明，记住就好。

什么是线段树

- 每次减半可以保证线段树层数在 $\mathcal{O}(\log n)$ 。
- 这样每次单点修改和区间询问就可以让时间复杂度稳定在 $\mathcal{O}(\log n)$ 。
- 线段树节点数可以证明不超过 $4n$ ，贴个[证明](#)，记住就好。
- 因为线段树只有最后一层可能有空，所以可以用顺序存储的方式，对于点 x ，左儿子是 $2x$ ，右儿子就是 $2x + 1$ 。

什么是线段树

- 每次减半可以保证线段树层数在 $\mathcal{O}(\log n)$ 。
- 这样每次单点修改和区间询问就可以让时间复杂度稳定在 $\mathcal{O}(\log n)$ 。
- 线段树节点数可以证明不超过 $4n$ ，贴个证明，记住就好。
- 因为线段树只有最后一层可能有空，所以可以用顺序存储的方式，对于点 x ，左儿子是 $2x$ ，右儿子就是 $2x + 1$ 。
- 实现如下：

线段树

```
11 a[N],t[N<<2];  
void push_up(int rt){t[rt]=t[rt<<1]+t[rt<<1|1];} // 合并区间信息  
void build(int l,int r,int rt){ // 初始化  
    if(l==r) return t[rt]=a[l],void(); // 赋值叶子信息  
    int mid=(l+r)>>1;  
    build(l,mid,rt<<1,mid+1,r,rt<<1|1);  
    push_up(rt);  
}
```


线段树

```
#define qurey query
ll query(int L,int R,int l,int r,int rt){
    if(L<=l&&R<=r)return t[rt];//若当前区间为子区间直接返回答案
    int mid=(l+r)>>1;ll res=0;
    if(L<=mid)res+=query(L,R,l,mid,rt<<1);
    if(R>mid) res+=qurey(L,R,mid+1,r,rt<<1|1);
    return res;//答案一定是一个前缀和一个后缀拼起来
}//询问区间和

void update(int x,ll k,int l,int r,int rt){
    if(l==r)return t[rt]+=k,void();
    int mid=(l+r)>>1;
    if(x<=mid)update(x,k,l,mid,rt<<1);
    else update(x,k,mid+1,r,rt<<1|1);
    push_up(rt);//修改后要更新区间信息
}//单点修改
```

牛一点的 push_up

- $(G, \cdot)$ 中的运算符还可以用 $\max, \min, \text{xor}$ 甚至矩阵等操作代替。

牛一点的 push_up

- $(G, \cdot)$ 中的运算符还可以用 $\max, \min, \text{xor}$ 甚至矩阵等操作代替。
- 有些 push_up 可以有一些特殊的操作, 比如 push_up 套 query。

牛一点的 push_up

- $(G, \cdot)$ 中的运算符还可以用 $\max, \min, \text{xor}$ 甚至矩阵等操作代替。
- 有些 push_up 可以有一些特殊的操作, 比如 push_up 套 query。
- 还可以是若干信息合并, 比如下面三道例题。

GSS3-Can you answer these queries III

- n 个数, q 次操作
- 操作 $0xy$ 把 a_x 修改为 y 。
- 操作 $1lr$ 询问区间 $[l, r]$ 的最大子段和。

GSS3-Can you answer these queries III

- 考虑区间合并。

GSS3-Can you answer these queries III

- 考虑区间合并。
- 单独每个区间，左区间的最大子段和，右区间的最大子段可能会影响答案。

GSS3-Can you answer these queries III

- 考虑区间合并。
- 单独每个区间，左区间的最大子段和，右区间的最大子段可能会影响答案。
- 接下来考虑跨越两个区间的最大子段和，可以发现，只有左区间的后缀最大子段和和右区间的前缀最大子段和会影响答案。

GSS3-Can you answer these queries III

- 考虑区间合并。
- 单独每个区间，左区间的最大子段和，右区间的最大子段可能会影响答案。
- 接下来考虑跨越两个区间的最大子段和，可以发现，只有左区间的后缀最大子段和和右区间的前缀最大子段和会影响答案。
- 所以线段树每个节点只需要维护前缀、后缀最大子段和和最大子段和就可以了。

Narrow Components

- 给定一个 $3 \times n$ 的 01 网格图，两个相邻的 1 联通。
多组询问，每组询问给定 l, r ，求 l 列至 r 列之间有多少个 1 构成的联通块。

Narrow Components

- 给定一个 $3 \times n$ 的 01 网格图，两个相邻的 1 联通。
多组询问，每组询问给定 l, r ，求 l 列至 r 列之间有多少个 1 构成的联通块。
- $1 \leq n \leq 5 \times 10^5, 1 \leq m \leq 3 \times 10^5$

Narrow Components

- 直接对列建线段树。

Narrow Components

- 直接对列建线段树。
- 每个节点存区间 $[l, r]$ 中的最左列 (a_l) 和最右列 (a_r) 的连通情况和颜色，只有这两个信息合并的时候会影响到答案。

Narrow Components

- 直接对列建线段树。
- 每个节点存区间 $[l, r]$ 中的最左列 (a_l) 和最右列 (a_r) 的连通情况和颜色，只有这两个信息合并的时候会影响到答案。
- 然后合并的时候直接用并查集维护，减掉出现联通的数量就可以了。

Narrow Components

- 直接对列建线段树。
- 每个节点存区间 $[l, r]$ 中的最左列 (a_l) 和最右列 (a_r) 的连通情况和颜色，只有这两个信息合并的时候会影响到答案。
- 然后合并的时候直接用并查集维护，减掉出现联通的数量就可以了。
- 时间复杂度 $\mathcal{O}(n \log n)$

楼房重建

- 有一片施工工地，每天，这片工地上的房子拆了又建、建了又拆(无不良引导)，问你从原点可以看见的楼房数。
- 形式化题意：前缀最长上升斜率长度。
- $1 \leq n, m \leq 10^5$

楼房重建

- 造房子，拆房子相当于每次修改一个斜率。

楼房重建

- 造房子，拆房子相当于每次修改一个斜率。
- 合并区间的时候，右儿子的答案和左儿子息息相关，所以每次 `push_up` 的时候都询问一遍右边比左边最高值高的最长前缀长度就可以了。

楼房重建

- 造房子，拆房子相当于每次修改一个斜率。
- 合并区间的时候，右儿子的答案和左儿子息息相关，所以每次 `push_up` 的时候都询问一遍右边比左边最高值高的最长前缀长度就可以了。
- 实现如下：

```
if(mx[rt<<1]<=k)return query(k,mid+1,r,rt<<1|1);
return query(k,l,mid,rt<<1)+t[rt]-t[rt<<1]; //询问操作
t[rt]=t[rt<<1]+query(mx[rt<<1],mid+1,r,rt<<1|1); //
    push_up
```

楼房重建

- 造房子，拆房子相当于每次修改一个斜率。
- 合并区间的时候，右儿子的答案和左儿子息息相关，所以每次 `push_up` 的时候都询问一遍右边比左边最高值高的最长前缀长度就可以了。
- 实现如下：

```
if(mx[rt<<1]<=k)return query(k,mid+1,r,rt<<1|1);  
return query(k,l,mid,rt<<1)+t[rt]-t[rt<<1]; //询问操作  
t[rt]=t[rt<<1]+query(mx[rt<<1],mid+1,r,rt<<1|1); //  
    push_up
```

- 时间复杂度 $\mathcal{O}(n \log^2 n)$

楼房重建

- 造房子，拆房子相当于每次修改一个斜率。
- 合并区间的时候，右儿子的答案和左儿子息息相关，所以每次 `push_up` 的时候都询问一遍右边比左边最高值高的最长前缀长度就可以了。
- 实现如下：

```
if(mx[rt<<1]<=k)return query(k,mid+1,r,rt<<1|1);  
return query(k,l,mid,rt<<1)+t[rt]-t[rt<<1]; //询问操作  
t[rt]=t[rt<<1]+query(mx[rt<<1],mid+1,r,rt<<1|1); //  
    push_up
```

- 时间复杂度 $\mathcal{O}(n \log^2 n)$
- 注意：**千万不要忘记合并线段树信息的复杂度。

懒标记

- 因为线段树太勤劳了，每次都要从上向下修改直到叶子节点，以至于在修改区间的时候时间复杂度到了 $\mathcal{O}(n)$ 。

懒标记

- 因为线段树太勤劳了，每次都要从上向下修改直到叶子节点，以至于在修改区间的时候时间复杂度到了 $\mathcal{O}(n)$ 。
- 我们需要一种方法能让时间复杂度降到 $\mathcal{O}(\log n)$ ，于是就有了懒标记。

懒标记

- 因为线段树太勤劳了，每次都要从上向下修改直到叶子节点，以至于在修改区间的时候时间复杂度到了 $\mathcal{O}(n)$ 。
- 我们需要一种方法能让时间复杂度降到 $\mathcal{O}(\log n)$ ，于是就有了懒标记。
- 考虑修改整个区间的时候，到达对应区间的时候就停止向下遍历，直接在这个区间留下一个标记，表示当前被修改过。这样在每次询问的时候再下传标记就能让时间复杂度有效地下降。

懒标记

- 因为线段树太勤劳了，每次都要从上向下修改直到叶子节点，以至于在修改区间的时候时间复杂度到了 $\mathcal{O}(n)$ 。
- 我们需要一种方法能让时间复杂度降到 $\mathcal{O}(\log n)$ ，于是就有了懒标记。
- 考虑修改整个区间的时候，到达对应区间的时候就停止向下遍历，直接在这个区间留下一个标记，表示当前被修改过。这样在每次询问的时候再下传标记就能让时间复杂度有效地下降。
- 实现如下：

```
void f(int rt,int k){t[rt]+=k,lzy[rt]+=k;}//标记修改
void push_down(int rt){
    if(!lzy[rt])return;//为空不用下传
    f(rt<<1,lzy[rt]),f(rt<<1|1,lzy[rt]);
}//标记下传
```

- 每次询问和修改在向下递归之前都可以 `push_down` 一次。

- 每次询问和修改在向下递归之前都可以 `push_down` 一次。
- 有些情况在修改的时候不需要下传，但是询问的时候一定要下传。

- 每次询问和修改在向下递归之前都可以 `push_down` 一次。
- 有些情况在修改的时候不需要下传，但是询问的时候一定要下传。
- 还有永久标记不下传，太懒了，每次询问的时候都计算一遍懒标记的贡献，也就是“我们的红太阳”中提及到的算法。

动态开点线段树

- 有些时候值域非常大，你懒得写离散化，或者强制在线的时候你可以用动态开点的方法。

动态开点线段树

- 有些时候值域非常大，你懒得写离散化，或者强制在线的时候你可以用动态开点的方法。
- 具体实现是在使用的时候再向下开点。

动态开点线段树

- 有些时候值域非常大，你懒得写离散化，或者强制在线的时候你可以用动态开点的方法。
- 具体实现是在使用的时候再向下开点。
- 实现如下：

```
void update(int x,int k,int l,int r,int&rt=rot){
    if(!rt)rt=++cnt;
    if(l==r)return t[rt]++,void();
    int mid=(l+r)>>1;
    if(x<=mid)update(x,k,l,mid,ls[rt]);
    else update(x,k,mid+1,r,rs[rt]);
    push_up(rt);
}
```

动态开点线段树

- 有些时候值域非常大，你懒得写离散化，或者强制在线的时候你可以用动态开点的方法。
- 具体实现是在使用的时候再向下开点。
- 实现如下：

```
void update(int x,int k,int l,int r,int&rt=rot){
    if(!rt)rt=++cnt;
    if(l==r)return t[rt]++,void();
    int mid=(l+r)>>1;
    if(x<=mid)update(x,k,l,mid,ls[rt]);
    else update(x,k,mid+1,r,rs[rt]);
    push_up(rt);
}
```

- 一次修改最多增加 $\log w$ 个节点，空间开 $n \log w$ 就可以了，所以时空都是 $\mathcal{O}(n \log w)$ 的。

动态开点线段树

- 有些时候值域非常大，你懒得写离散化，或者强制在线的时候你可以用动态开点的方法。
- 具体实现是在使用的时候再向下开点。
- 实现如下：

```
void update(int x,int k,int l,int r,int&rt=rot){
    if(!rt)rt=++cnt;
    if(l==r)return t[rt]++,void();
    int mid=(l+r)>>1;
    if(x<=mid)update(x,k,l,mid,ls[rt]);
    else update(x,k,mid+1,r,rs[rt]);
    push_up(rt);
}
```

- 一次修改最多增加 $\log w$ 个节点，空间开 $n \log w$ 就可以了，所以时空都是 $\mathcal{O}(n \log w)$ 的。
- 但是动态开点空间和时间都比较劣，不是很建议使用，本人被卡过至少 3 次。

线段树二分

- 比如你要在线段树找左边第一个小于 x 的值，你只需要维护最小值，然后如果左边小于 x 就往左边搜，否则搜右边就可以了。

线段树二分

- 比如你要在线段树找左边第一个小于 x 的值，你只需要维护最小值，然后如果左边小于 x 就往左边搜，否则搜右边就可以了。
- 实现如下：

```
int find(int L,int R,int x,int l,int r,int rt){
    if(t[rt]>x)return -1;
    if(l==r)return l;
    int mid=(l+r)>>1;
    if(L<=mid&& t[rt<<1]<x)return find(L,R,x,l,mid,rt<<1);
    if(R>mid)return find(L,R,x,mid+1,r,rt<<1|1);
    return -1;
}
```

线段树分治

- 通过线段树的结构，对时间进行分治，从而在 $\mathcal{O}(n \log n)$ 的时间中维护出每一次询问的状态。

线段树分治

- 通过线段树的结构, 对时间进行分治, 从而在 $\mathcal{O}(n \log n)$ 的时间中维护出每一次询问的状态。
- 是一种离线算法, 如果不带修改, 一般也可以用 CDQ 分治替换。

线段树分治

- 通过线段树的结构，对时间进行分治，从而在 $\mathcal{O}(n \log n)$ 的时间中维护出每一次询问的状态。
- 是一种离线算法，如果不带修改，一般也可以用 CDQ 分治替换。
- 可以通过左端点加入，右端点放出来完成一些离线操作。

最小 mex 生成树

- n 个点 m 条边的无向连通图，边有边权。
- 求这个图最小的 mex 生成树。
- mex: 最小没有出现的自然数。
- $1 \leq n \leq 10^6, 1 \leq m \leq 2 \times 10^6, 0 \leq w \leq 10^5$

最小 mex 生成树

- 发现值域从左往右扫第一个即使不出现图也可以联通的点，就是答案。

最小 mex 生成树

- 发现值域从左往右扫第一个即使不出现图也可以联通的点，就是答案。
- 当答案为 x 的时候, $[0, x - 1] \cup [x + 1, +\infty)$ 都是可以有边存在的。

最小 mex 生成树

- 发现值域从左往右扫第一个即使不出现图也可以联通的点，就是答案。
- 当答案为 x 的时候， $[0, x - 1] \cup [x + 1, +\infty)$ 都是可以有边存在的。
- 所以线段树维护当前值为答案时可以存在的边，所以只要有区间加一条边操作就可以了。

最小 mex 生成树

- 发现值域从左往右扫第一个即使不出现图也可以联通的点，就是答案。
- 当答案为 x 的时候， $[0, x - 1] \cup [x + 1, +\infty)$ 都是可以有边存在的。
- 所以线段树维护当前值为答案时可以存在的边，所以只要有区间加一条边操作就可以了。
- 然后从左往右扫描线段树，进入区间的时候往并查集里插入边，退出区间的时候撤销就可以了，判断是否可行就是是否全联通。

最小 mex 生成树

- 发现值域从左往右扫第一个即使不出现图也可以联通的点，就是答案。
- 当答案为 x 的时候， $[0, x - 1] \cup [x + 1, +\infty)$ 都是可以有边存在的。
- 所以线段树维护当前值为答案时可以存在的边，所以只要有区间加一条边操作就可以了。
- 然后从左往右扫描线段树，进入区间的时候往并查集里插入边，退出区间的时候撤销就可以了，判断是否可行就是是否全联通。
- 时间复杂度 $\mathcal{O}(w \log w \log n)$

权值线段树

- 和权值树状数组类似，对权值建一颗线段树。

权值线段树

- 和权值树状数组类似，对权值建一颗线段树。
- 同样的，它可以快速求出 k th。

权值线段树

- 和权值树状数组类似，对权值建一颗线段树。
- 同样的，它可以快速求出 kth 。
- 它还支持时间复杂度较低的合并和分裂，基于动态开点。

合并

- 把 y 合并给 x , 直接把 y 的所有点累加给 x , 如果 x 没有这个节点直接赋值为 y 。

合并

- 把 y 合并给 x , 直接把 y 的所有点累加给 x , 如果 x 没有这个节点直接赋值为 y 。
- 避免空间浪费可以写一个回收站来回收 y 中没用的节点。

合并

- 把 y 合并给 x , 直接把 y 的所有点累加给 x , 如果 x 没有这个节点直接赋值为 y 。
- 避免空间浪费可以写一个回收站来回收 y 中没用的节点。
- 实现如下:

```
void del(int x){rub[++tot]=x,ls[x]=rs[x]=t[x]=0;}
int merge(int x,int y){
    if(!x||!y)return x^y;//直接返回其中一个
    t[x]+=t[y];
    ls[x]=merge(ls[x],rs[y]);
    rs[x]=merge(rs[x],rs[y]);
    return del(x),x;
}
```

合并

- 把 y 合并给 x , 直接把 y 的所有点累加给 x , 如果 x 没有这个节点直接赋值为 y 。
- 避免空间浪费可以写一个回收站来回收 y 中没用的节点。
- 实现如下:

```
void del(int x){rub[++tot]=x,ls[x]=rs[x]=t[x]=0;}
int merge(int x,int y){
    if(!x||!y)return x^y;//直接返回其中一个
    t[x]+=t[y];
    ls[x]=merge(ls[x],rs[y]);
    rs[x]=merge(rs[x],rs[y]);
    return del(x),x;
}
```

- 合并均摊时间复杂度是 $\mathcal{O}(\log n)$, 只有合并可以直接分析最劣情况得到是 $\mathcal{O}(n \log n)$ 。

合并

- 把 y 合并给 x , 直接把 y 的所有点累加给 x , 如果 x 没有这个节点直接赋值为 y 。
- 避免空间浪费可以写一个回收站来回收 y 中没用的节点。
- 实现如下:

```
void del(int x){rub[++tot]=x,ls[x]=rs[x]=t[x]=0;}
int merge(int x,int y){
    if(!x||!y)return x^y;//直接返回其中一个
    t[x]+=t[y];
    ls[x]=merge(ls[x],rs[y]);
    rs[x]=merge(rs[x],rs[y]);
    return del(x),x;
}
```

- 合并均摊时间复杂度是 $\mathcal{O}(\log n)$, 只有合并可以直接分析最劣情况得到是 $\mathcal{O}(n \log n)$ 。



* 分裂

- 首先看 x 的左子树的值 v , 如果 $v < k$, 那么左边归 x , 递归右边即可, 注意 k 变成了 $k - v$ 。
如果 $v = k$, 那么左边归 x , 右边归 y 。
如果 $v > k$, 那么右边归 y , 递归左边。
 x 的新权值是 k , 那么 y 的权值就是 x 原先的权值减去 k 了。

* 分裂

- 首先看 x 的左子树的值 v , 如果 $v < k$, 那么左边归 x , 递归右边即可, 注意 k 变成了 $k - v$.
如果 $v = k$, 那么左边归 x , 右边归 y .
如果 $v > k$, 那么右边归 y , 递归左边.
 x 的新权值是 k , 那么 y 的权值就是 x 原先的权值减去 k 了.
- 实现如下:

```
inline void split(int x,int&y,ll k){  
    if(!x)return;  
    nw(y);  
    ll g=t[ls[x]];   
    if(k>g)split(rs[x],rs[y],k-g);  
    else swap(rs[x],rs[y]); //x 给 y  
    if(k<g)split(ls[x],ls[y],k);  
    t[y]=t[x]-k,t[x]=k;  
    return;  
}
```

* 分裂

- 和 kth 差不多, 时间复杂度是 $\mathcal{O}(\log n)$

* 分裂

- 和 kth 差不多，时间复杂度是 $\mathcal{O}(\log n)$
- 同时合并和分裂的时间复杂度可以证明，具体看[这里](#)，就不展开了。

历史最值

- 操作：在每一次操作后，都进行一次更新，让 $b_i \leftarrow \max(b_i, a_i)$ ， b_i 就是历史最值。

历史最值

- 操作：在每一次操作后，都进行一次更新，让 $b_i \leftarrow \max(b_i, a_i)$ ， b_i 就是历史最值。
- 对于有加法的历史最值问题，只需要考虑当前最大值和历史最大加就可以顺利地求出历史最大值了。

历史最值

- 操作：在每一次操作后，都进行一次更新，让 $b_i \leftarrow \max(b_i, a_i)$ ， b_i 就是历史最值。
- 对于有加法的历史最值问题，只需要考虑当前最大值和历史最大加就可以顺利地求出历史最大值了。
- `push_down` 的时候历史最大加可以更新为当前加 + 下传的历史最大加。

历史最值

- 操作：在每一次操作后，都进行一次更新，让 $b_i \leftarrow \max(b_i, a_i)$ ， b_i 就是历史最值。
- 对于有加法的历史最值问题，只需要考虑当前最大值和历史最大加就可以顺利地求出历史最大值了。
- `push_down` 的时候历史最大加可以更新为当前加 + 下传的历史最大加。
- 然后就能愉快地维护了。

- 长度为 n 的序列 a , 进行 m 次以下四种操作:
 1. 求 $[l, r]$ 的最大值。
 2. 求 $[l, r]$ 的历史最大值。
 3. 将 $[l, r]$ 加 x 。
 4. 将 $[l, r]$ 赋值为 x 。
- $1 \leq n, m \leq 10^5$

- 难点在赋值操作。

- 难点在赋值操作。
- 发现一次赋值以后所有加操作都可以用赋值操作代替了。

- 难点在赋值操作。
- 发现一次赋值以后所有加操作都可以用赋值操作代替了。
- 也就是 $a \leftarrow a + k$ 。

- 难点在赋值操作。
- 发现一次赋值以后所有加操作都可以用赋值操作代替了。
- 也就是 $a \leftarrow a + k$ 。
- 同时更新历史值就可以了。

* 区间最值

- 操作：对于所有的 $i \in [l, r]$, 将 a_i 变成 $\min(a_i, v)$ 。

* 区间最值

- 操作：对于所有的 $i \in [l, r]$ ，将 a_i 变成 $\min(a_i, v)$ 。
- 线段树的每个节点维护三个信息：区间最大值 mx ，区间严格次大值 se 和最大值的个数 cnt 。那么，一次区间最值操作作用在这个节点上时，可以被分为以下三种情况：

* 区间最值

- 操作：对于所有的 $i \in [l, r]$ ，将 a_i 变成 $\min(a_i, v)$ 。
- 线段树的每个节点维护三个信息：区间最大值 mx ，区间严格次大值 se 和最大值的个数 cnt 。那么，一次区间最值操作作用在这个节点上时，可以被分为以下三种情况：
- $k \geq mx$ ，没有点要操作，直接退出。

* 区间最值

- 操作：对于所有的 $i \in [l, r]$ ，将 a_i 变成 $\min(a_i, v)$ 。
- 线段树的每个节点维护三个信息：区间最大值 mx ，区间严格次大值 se 和最大值的个数 cnt 。那么，一次区间最值操作作用在这个节点上时，可以被分为以下三种情况：
- $k \geq mx$ ，没有点要操作，直接退出。
- $se < k < mx$ ，区间内所有最大值都被修改为 k ，而最大值个数不变。将区间和加上 $cnt \cdot (k - mx)$ ，打上懒标记，退出。

* 区间最值

- 操作：对于所有的 $i \in [l, r]$ ，将 a_i 变成 $\min(a_i, v)$ 。
- 线段树的每个节点维护三个信息：区间最大值 mx ，区间严格次大值 se 和最大值的个数 cnt 。那么，一次区间最值操作作用在这个节点上时，可以被分为以下三种情况：
- $k \geq mx$ ，没有点要操作，直接退出。
- $se < k < mx$ ，区间内所有最大值都被修改为 k ，而最大值个数不变。将区间和加上 $cnt \cdot (k - mx)$ ，打上懒标记，退出。
- $k \leq se$ ，直接暴力往下修改。

* 区间最值

- 操作：对于所有的 $i \in [l, r]$ ，将 a_i 变成 $\min(a_i, v)$ 。
- 线段树的每个节点维护三个信息：区间最大值 mx ，区间严格次大值 se 和最大值的个数 cnt 。那么，一次区间最值操作作用在这个节点上时，可以被分为以下三种情况：
- $k \geq mx$ ，没有点要操作，直接退出。
- $se < k < mx$ ，区间内所有最大值都被修改为 k ，而最大值个数不变。将区间和加上 $cnt \cdot (k - mx)$ ，打上懒标记，退出。
- $k \leq se$ ，直接暴力往下修改。
- 当没有区间加的时候时间复杂度还是可以证明的：

* 区间最值

- 操作：对于所有的 $i \in [l, r]$ ，将 a_i 变成 $\min(a_i, v)$ 。
- 线段树的每个节点维护三个信息：区间最大值 mx ，区间严格次大值 se 和最大值的个数 cnt 。那么，一次区间最值操作作用在这个节点上时，可以被分为以下三种情况：
- $k \geq mx$ ，没有点要操作，直接退出。
- $se < k < mx$ ，区间内所有最大值都被修改为 k ，而最大值个数不变。将区间和加上 $cnt \cdot (k - mx)$ ，打上懒标记，退出。
- $k \leq se$ ，直接暴力往下修改。
- 当没有区间加的时候时间复杂度还是可以证明的：
- 每次向下操作必定能使序列内的数字类数 -1 ，线段树从上到下每个节点最多遍历 $\mathcal{O}(n \log n)$ 个节点，加上每次询问向下的时间复杂度就是 $\mathcal{O}((n + m) \log n)$ 。

* 区间最值

- 操作：对于所有的 $i \in [l, r]$ ，将 a_i 变成 $\min(a_i, v)$ 。
- 线段树的每个节点维护三个信息：区间最大值 mx ，区间严格次大值 se 和最大值的个数 cnt 。那么，一次区间最值操作作用在这个节点上时，可以被分为以下三种情况：
- $k \geq mx$ ，没有点要操作，直接退出。
- $se < k < mx$ ，区间内所有最大值都被修改为 k ，而最大值个数不变。将区间和加上 $cnt \cdot (k - mx)$ ，打上懒标记，退出。
- $k \leq se$ ，直接暴力往下修改。
- 当没有区间加的时候时间复杂度还是可以证明的：
- 每次向下操作必定能使序列内的数字类数 -1 ，线段树从上到下每个节点最多遍历 $\mathcal{O}(n \log n)$ 个节点，加上每次询问向下的时间复杂度就是 $\mathcal{O}((n + m) \log n)$ 。
- 但是有区间加操作只能证明到上界是 $\mathcal{O}(\log^2 n)$ ，就不展开了。

均摊复杂度

- 均摊大部分情况可以用势能分析证明，我这里为了便于理解就直接感性理解了。

- 均摊大部分情况可以用势能分析证明，我这里为了便于理解就直接感性理解了。
- 势能还是建议学一下，虽然没什么用，考场上也想不到。

GSS4-Can you answer these queries IV

- 操作 0: 把区间 $[x, y]$ 的每个数开根, 向下取整。
- 操作 1: 询问区间 $[x, y]$ 和。
- $1 \leq n \leq 10^5, 1 \leq a_i \leq 10^{18}$

GSS4-Can you answer these queries IV

- 可以发现, 没开几次, a_i 就变成 1 了。

GSS4-Can you answer these queries IV

- 可以发现, 没开几次, a_i 就变成 1 了。
- 继续考虑发现每个数最多只会被开 6 次根号。

GSS4-Can you answer these queries IV

- 可以发现, 没开几次, a_i 就变成 1 了。
- 继续考虑发现每个数最多只会被开 6 次根号。
- 那一共只会操作 $6 \times n$ 。

GSS4-Can you answer these queries IV

- 可以发现, 没开几次, a_i 就变成 1 了。
- 继续考虑发现每个数最多只会被开 6 次根号。
- 那一共只会操作 $6 \times n$ 。
- 时间复杂度 $\mathcal{O}((n + m) \log n)$ 。

GSS4-Can you answer these queries IV

- 可以发现, 没开几次, a_i 就变成 1 了。
- 继续考虑发现每个数最多只会被开 6 次根号。
- 那一共只会操作 $6 \times n$ 。
- 时间复杂度 $\mathcal{O}((n + m) \log n)$ 。
- 如果再加上区间加操作, 可以通过极差 ≤ 1 的区间剪枝, 因为根号能让两个极差大于 1 的数在很少的次数内使极差 ≤ 1 。

GSS4-Can you answer these queries IV

- 可以发现，没开几次， a_i 就变成 1 了。
- 继续考虑发现每个数最多只会被开 6 次根号。
- 那一共只会操作 $6 \times n$ 。
- 时间复杂度 $\mathcal{O}((n + m) \log n)$ 。
- 如果再加上区间加操作，可以通过极差 ≤ 1 的区间剪枝，因为根号能让两个极差大于 1 的数在很少的次数内使极差 ≤ 1 。
- 对于极差 = 1 举例： $[2, 3, 2, 3] + 6$ 操作，sqrt 操作不断循环。

- 类似的，取模，取 φ 之类每次减半的都能直接做，时间复杂度是 $\mathcal{O}(n \log^2 n)$ 。

- 对于扫描，可以理解为对时间轴（就是顺序做），数字轴.....

扫描

- 对于扫描，可以理解为对时间轴（就是顺序做），数字轴.....
- 但是有一种扫描线扫二维坐标系，就是扫描线算法。

扫描

- 对于扫描，可以理解为对时间轴（就是顺序做），数字轴.....
- 但是有一种扫描线扫二维坐标系，就是扫描线算法。
- 具体思想就是加入一条平行于 x 或者 y 轴的直线，从最下面开始往上扫描，扫描的时候对另一条轴建立坐标系进行操作。

扫描

- 对于扫描，可以理解为对时间轴（就是顺序做），数字轴.....
- 但是有一种扫描线扫二维坐标系，就是扫描线算法。
- 具体思想就是加入一条平行于 x 或者 y 轴的直线，从最下面开始往上扫描，扫描的时候对另一条轴建立坐标系进行操作。
- 这样就可以在数据结构维护的情况下做到 $\mathcal{O}(n \log n)$ 的复杂度了。

【模板】扫描线

- 求 n 个矩形的并集覆盖的总面积，边平行于 x/y 轴。
- $1 \leq n \leq 10^5$

【模板】扫描线

- 当时有人问过我具体实现，但是当时没有给出具体回答，自己也糊里糊涂的，现在解答一下以表歉意 @lanos212

【模板】扫描线

- 当时有人问过我具体实现，但是当时没有给出具体回答，自己也糊里糊涂的，现在解答一下以表歉意 @lanos212
- 这里建平行于 x 轴的直线从下往上扫。具体实现可以把矩形分成两条线段，一条是下面的线段加入，一条是上面的线段删除。

【模板】扫描线

- 当时有人问过我具体实现，但是当时没有给出具体回答，自己也糊里糊涂的，现在解答一下以表歉意 @lanos212
- 这里建平行于 x 轴的直线从下往上扫。具体实现可以把矩形分成两条线段，一条是下面的线段加入，一条是上面的线段删除。
- 然后问题就变成了往线段树里加入删除线段并统计 Σ 。

【模板】扫描线

- 当时有人问过我具体实现，但是当时没有给出具体回答，自己也糊里糊涂的，现在解答一下以表歉意 @lanos212
- 这里建平行于 x 轴的直线从下往上扫。具体实现可以把矩形分成两条线段，一条是下面的线段加入，一条是上面的线段删除。
- 然后问题就变成了往线段树里加入删除线段并统计 Σ 。
- 由于一个点会被多次覆盖，所以需要记录一个懒标记记录被覆盖几次，防止删除的时候全删了。

【模板】扫描线

- 当时有人问过我具体实现，但是当时没有给出具体回答，自己也糊里糊涂的，现在解答一下以表歉意 @lanos212
- 这里建平行于 x 轴的直线从下往上扫。具体实现可以把矩形分成两条线段，一条是下面的线段加入，一条是上面的线段删除。
- 然后问题就变成了往线段树里加入删除线段并统计 Σ 。
- 由于一个点会被多次覆盖，所以需要记录一个懒标记记录被覆盖几次，防止删除的时候全删了。
- 怎么统计答案呢，设一个 t , $t_x = t_{ls_x} + t_{rs_x}$ 是当前区间没有被覆盖的情况，也就是 $lzy_x > 0$ ；否则说明全部覆盖，直接等于区间长度就可以了。最后 t_{rot} 就是答案。

【模板】扫描线

- 当时有人问过我具体实现，但是当时没有给出具体回答，自己也糊里糊涂的，现在解答一下以表歉意 @lanos212
- 这里建平行于 x 轴的直线从下往上扫。具体实现可以把矩形分成两条线段，一条是下面的线段加入，一条是上面的线段删除。
- 然后问题就变成了往线段树里加入删除线段并统计 Σ 。
- 由于一个点会被多次覆盖，所以需要记录一个懒标记记录被覆盖几次，防止删除的时候全删了。
- 怎么统计答案呢，设一个 t , $t_x = t_{ls_x} + t_{rs_x}$ 是当前区间没有被覆盖的情况，也就是 $lzy_x > 0$ ；否则说明全部覆盖，直接等于区间长度就可以了。最后 t_{rot} 就是答案。
- 为什么是对的？考虑修改一个区间，只可能会对这个区间的 t 产生影响，对别的区间的 t 没有影响，所以只需要减到 0 或者 > 0 了就修改一下就可以了。

* 奇奇怪怪的芝士

- 还有一些特殊的线段树比如：

* 奇奇怪怪的芝士

- 还有一些特殊的线段树比如：
- 猫树 (不修改来做到单次询问 $\mathcal{O}(1)$)。

* 奇奇怪怪的芝士

- 还有一些特殊的线段树比如：
- 猫树 (不修改来做到单次询问 $\mathcal{O}(1)$)。
- 李超树(可以维护多条线段最值的树，留给老莽讲了)。

* 奇奇怪怪的芝士

- 还有一些特殊的线段树比如：
- 猫树 (不修改来做到单次询问 $\mathcal{O}(1)$)。
- 李超树(可以维护多条线段最值的树，留给老莽讲了)。
- 线段树套线段树(二维线段树，大多用动态开点实现来节省空间)。

* 奇奇怪怪的芝士

- 还有一些特殊的线段树比如：
- 猫树 (不修改来做到单次询问 $\mathcal{O}(1)$)。
- 李超树(可以维护多条线段最值的树，留给老莽讲了)。
- 线段树套线段树(二维线段树，大多用动态开点实现来节省空间)。
- 线段树优化建图(区间之间最短路建图方法)。

* 奇奇怪怪的芝士

- 还有一些特殊的线段树比如：
- 猫树 (不修改来做到单次询问 $\mathcal{O}(1)$)。
- 李超树(可以维护多条线段最值的树，留给老莽讲了)。
- 线段树套线段树(二维线段树，大多用动态开点实现来节省空间)。
- 线段树优化建图(区间之间最短路建图方法)。
- 跳表($\mathcal{O}(\log n)$ 的链表)。

目录

- 1 ST 表
- 2 树状数组
- 3 线段树
- 4 珂朵莉树
 - ODT
 - 例题
- 5 可持久化线段树 (主席树)
- 6 我们的红太阳

什么是 ODT

- 珂朵莉（又名老司机树，简称 ODT，还叫颜色段均摊）是来自CF896C 的暴力数据结构，原理是同时维护所有值相同的元素来达到较低的时间复杂度。其保证时间复杂度的操作是区间推平，没有足够多而且足够大的区间推平的话珂朵莉的时间复杂度就是假的。

什么是 ODT

- 珂朵莉（又名老司机树，简称 ODT，还叫颜色段均摊）是来自CF896C 的暴力数据结构，原理是同时维护所有值相同的元素来达到较低的时间复杂度。其保证时间复杂度的操作是区间推平，没有足够多而且足够大的区间推平的话珂朵莉的时间复杂度就是假的。
- split 的作用是把一个区间 $[l, r]$ 分成 $[l, p]$ 和 $[p + 1, r]$

什么是 ODT

- 珂朵莉（又名老司机树，简称 ODT，还叫颜色段均摊）是来自CF896C 的暴力数据结构，原理是同时维护所有值相同的元素来达到较低的时间复杂度。其保证时间复杂度的操作是区间推平，没有足够多而且足够大的区间推平的话珂朵莉的时间复杂度就是假的。
- split 的作用是把一个区间 $[l, r]$ 分成 $[l, p]$ 和 $[p + 1, r]$
- assign 的作用是把 $[l, r]$ 的区间推平，直接 erase $[l, r]$ 的迭代器就行了。这里注意 split 的顺序一定要先 split $r + 1$ （因为 STL 是左闭右开的所以要多取一位，这也是为什么珂朵莉刚开始插入初始区间要 insert $n + 1$ 的原因）再 split l 否则会 RE。

什么是 ODT

- 珂朵莉 (又名老司机树, 简称 ODT, 还叫颜色段均摊) 是来自 CF896C 的暴力数据结构, 原理是同时维护所有值相同的元素来达到较低的时间复杂度。其保证时间复杂度的操作是区间推平, 没有足够多而且足够大的区间推平的话珂朵莉的时间复杂度就是假的。
- split 的作用是把一个区间 $[l, r]$ 分成 $[l, p]$ 和 $[p + 1, r]$
- assign 的作用是把 $[l, r]$ 的区间推平, 直接 erase $[l, r]$ 的迭代器就行了。这里注意 split 的顺序一定要先 split $r + 1$ (因为 STL 是左闭右开的所以要多取一位, 这也是为什么珂朵莉刚开始插入初始区间要 insert $n + 1$ 的原因) 再 split l 否则会 RE。
- 因为如果 split l , 那么 l 的迭代器可能会在 split $r + 1$ 的时候被再次分裂导致 l 的迭代器失效。但是如果先 split $r + 1$, 可以保证 l 的迭代器肯定在 $r + 1$ 的左边不会对 $r + 1$ 生效。

- 可以用 list 或者 set 实现。

- 可以用 list 或者 set 实现。
- list 只能保证在随机数据下时间复杂度正确，而 set 则可以保证在均摊下时间复杂度正确。

- 可以用 list 或者 set 实现。
- list 只能保证在随机数据下时间复杂度正确，而 set 则可以保证在均摊下时间复杂度正确。
- 随机数据下 list 实现时间复杂度 $\mathcal{O}(n \log n)$, set 实现时间复杂度 $\mathcal{O}(n \log \log n)$ 。

- 可以用 list 或者 set 实现。
- list 只能保证在随机数据下时间复杂度正确，而 set 则可以保证在均摊下时间复杂度正确。
- 随机数据下 list 实现时间复杂度 $\mathcal{O}(n \log n)$, set 实现时间复杂度 $\mathcal{O}(n \log \log n)$ 。
- 在不随机的有些区间操作可以直接用珂朵莉维护，具体就是一些整个区间一起进行的操作，比如区间交换，反转之类的东西。

- 可以用 list 或者 set 实现。
- list 只能保证在随机数据下时间复杂度正确，而 set 则可以保证在均摊下时间复杂度正确。
- 随机数据下 list 实现时间复杂度 $\mathcal{O}(n \log n)$, set 实现时间复杂度 $\mathcal{O}(n \log \log n)$ 。
- 在不随机的有些区间操作可以直接用珂朵莉维护，具体就是一些整个区间一起进行的操作，比如区间交换，反转之类的东西。
- 推本校优秀选手论文：[珂朵莉进阶](#)

- 实现如下:

```

struct node{
    int l,r;mutable ll val;
    node(int L=-1,int R=-1,ll V=0){l=L,r=R,val=V;}
    bool operator<(const node&a)const{return l<a.l;}
};set<node>st;
vector<node>::iterator split(int pos){
    auto it=st.lower_bound(node(pos));
    if(it!=st.end()&&it->l==pos)return it;
    it--;node tmp=*it;st.erase(it);
    st.insert(node(tmp.l,pos-1,tmp.val));
    return st.insert(node{pos,tmp.r,tmp.val}).first;
}
void assign(int l,int r,ll val){
    auto itr=split(r+1),itl=split(l);
    st.erase(itl,itr),st.insert(node(l,r,val));
}

```

- 维护一种奇怪的数据结构，支持：
 1. 将 $[l, r]$ 区间所有数加上 x
 2. 将 $[l, r]$ 区间所有数改成 x
 3. 输出将 $[l, r]$ 区间第 x 小 ($1 \leq x \leq r - l + 1$)
 4. 输出 $[l, r]$ 区间每个数字的 x 次方的和模 y (每次询问不同) 的值
即

$$\sum_{i=l}^r a_i^n \pmod{y}$$

- $1 \leq n \leq 10^5$, 数据随机

- 板子题，因为是随机数据，所以用直接 list 或者 set 维护。

- 板子题，因为是随机数据，所以用直接 list 或者 set 维护。
- kth 直接把所有点拿出来排序然后直接扫就可以了。

- 板子题，因为是随机数据，所以用直接 list 或者 set 维护。
- kth 直接把所有点拿出来排序然后直接扫就可以了。
- 具体实现：
list, set。

- 若干操作：
 1. 求区间和
 2. 区间赋值
 3. 区间加
 4. 将 $[l_1, r_1]$ 复制到 $[l_2, r_2]$
 5. 交换 $[l_1, r_1]$ 和 $[l_2, r_2]$
 6. 反转 $[l, r]$
- $1 \leq 3 \times 10^5, r_1 - l_1 = r_2 - l_2, [l_1, r_1] \cap [l_2, r_2] = \emptyset$, 数据随机

- 很明显要干什么，有问题直接问吧。

目录

- 1 ST 表
- 2 树状数组
- 3 线段树
- 4 珂朵莉树
- 5 可持久化线段树 (主席树)
 - 可持久化
 - 主席树
 - 例题
- 6 我们的红太阳

什么是可持久化

- 可持久化数据结构：总是可以保留每一个历史版本，并且支持操作的不可变特性。

什么是可持久化

- 可持久化数据结构：总是可以保留每一个历史版本，并且支持操作的不可变特性。
- 部分可持久化算法：所有版本都可以访问，但是只有最新版本可以修改。

什么是可持久化

- 可持久化数据结构：总是可以保留每一个历史版本，并且支持操作的不可变特性。
- 部分可持久化算法：所有版本都可以访问，但是只有最新版本可以修改。
- 完全可持久化：所有版本都既可以访问又可以修改，若支持将两个历史版本合并，则又称为 Confluently Persistent。

什么是主席树

- 主席树就是部分可持久化的线段树。

什么是主席树

- 主席树就是部分可持久化的线段树。
- 看下面这道题目：求区间 $[l, r]$ 的 k th。

什么是主席树

- 主席树就是部分可持久化的线段树。
- 看下面这道题目：求区间 $[l, r]$ 的 k th。
- 用主席树保存每次插入操作的历史版本，来查询 k th。

什么是主席树

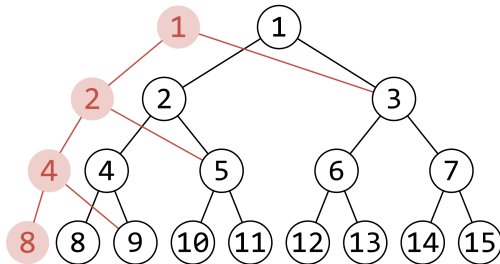
- 主席树就是部分可持久化的线段树。
- 看下面这道题目：求区间 $[l, r]$ 的 k th。
- 用主席树保存每次插入操作的历史版本，来查询 k th。
- 相当于每次询问的时候用 r 版本减掉 $l - 1$ 版本就是区间 $[l, r]$ ，然后和普通线段树一样二分就可以了。

什么是主席树

- 主席树就是部分可持久化的线段树。
- 看下面这道题目：求区间 $[l, r]$ 的 k th。
- 用主席树保存每次插入操作的历史版本，来查询 k th。
- 相当于每次询问的时候用 r 版本减掉 $l - 1$ 版本就是区间 $[l, r]$ ，然后和普通线段树一样二分就可以了。
- 保存历史版本的时候显然不能把所有节点都复制一遍，空间爆炸。

什么是主席树

- 主席树就是部分可持久化的线段树。
- 看下面这道题目：求区间 $[l, r]$ 的 k th。
- 用主席树保存每次插入操作的历史版本，来查询 k th。
- 相当于每次询问的时候用 r 版本减掉 $l - 1$ 版本就是区间 $[l, r]$ ，然后和普通线段树一样二分就可以了。
- 保存历史版本的时候显然不能把所有节点都复制一遍，空间爆炸。
- 考虑每次插入操作影响到的节点，只有 $\log n$ 个，如下图：



主席树

- 于是你就有了一个可以静态询问区间 $[l, r]$ 之间内容的数据结构。

主席树

- 于是你就有了一个可以静态询问区间 $[l, r]$ 之间内容的数据结构。
- 实现如下：

```
void insert(int pre,int k,int l,int r,int&rt){
    rt=++cnt;
    ls[rt]=ls[pre],rs[rt]=rs[pre],t[rt]=t[pre]+1;
    if(l==r)return;
    int mid=(l+r)>>1;
    if(k<=mid)insert(ls[pre],k,l,mid,ls[rt]);
    else insert(rs[pre],k,mid+1,r,rs[rt]);
} // 插入

int query(int u,int v,int k,int l,int r){
    int mid=(l+r)>>1,x=t[ls[v]]-t[ls[u]];
    if(l==r)return l;
    if(k<=x)return query(ls[u],ls[v],k,l,mid);
    else return query(rs[u],rs[v],k-x,mid+1,r);
} // 询问的时候减一下历史版本就好了
```

- 主席树的空间是 $n \log n$ 的，所以不要吝啬，开个 $32 \times n$ 差不多。

- 主席树的空间是 $n \log n$ 的，所以不要吝啬，开个 $32 \times n$ 差不多。
- 主席树一开始可以是一颗线段树，然后不断修改，所以会多一些空间别忘了。

主席树

- 主席树的空间是 $n \log n$ 的，所以不要吝啬，开个 $32 \times n$ 差不多。
- 主席树一开始可以是一颗线段树，然后不断修改，所以会多一些空间别忘了。
- 主席树有些时候可以用整体二分代替。

混合果汁

- 商店里有 n 种果汁, i 号果汁的美味度是 d_i , 每升价格为 p_i 。制作混合果汁时, 有一些特殊的规定: 在一瓶混合果汁中, i 号果汁最多只能添加 l_i 升。
- 现在有 m 个人。第 j 个人希望他得到的混合果汁总价格不大于 g_j , 体积不小于 L_j 。人们希望混合果汁的美味度尽可能高, 美味度等于所有参与混合的果汁的美味度的**最小值**。计算每个人能喝到的最美味的混合果汁的美味度。
- $1 \leq n, m \leq 10^5, 1 \leq d_i, p_i, l_i \leq 10^5, 1 \leq g_j, L_j \leq 10^{18}$

- 先可以贪心，把所有美味度从大到小排序，对总价格建线段树，同时维护一个体积，对美味度可持久化。

混合果汁

- 先可以贪心，把所有美味度从大到小排序，对总价格建线段树，同时维护一个体积，对美味度可持久化。
- 每次询问二分一个最大美味度，然后对一个前缀做区间差，从小到大在线段树上二分一个不小于 L_j 升的最小价格。

- 先可以贪心，把所有美味度从大到小排序，对总价格建线段树，同时维护一个体积，对美味度可持久化。
- 每次询问二分一个最大美味度，然后对一个前缀做区间差，从小到大在线段树上二分一个不小于 L_j 升的最小价格。
- 时间复杂度 $\mathcal{O}(n \log^2 n)$

混合果汁

- 先可以贪心，把所有美味度从大到小排序，对总价格建线段树，同时维护一个体积，对美味度可持久化。
- 每次询问二分一个最大美味度，然后对一个前缀做区间差，从小到大在线段树上二分一个不小于 L_j 升的最小价格。
- 时间复杂度 $\mathcal{O}(n \log^2 n)$
- 这里可以对每个美味度建主席树也可以对每个果汁建，虽然对美味度建可以优化空间，但是你还要记录一下时间戳，如果不是动态开空间的话没什么用。

可持久化并查集

- 维护一个并查集：
 1. 合并 a, b
 2. 回到第 k 次操作之后的状态
 3. 询问 a, b 是否联通
- $1 \leq n \leq 10^5, 1 \leq m \leq 10^5$

- 直接维护一个主席树表示 x 的祖先。

可持久化并查集

- 直接维护一个主席树表示 x 的祖先。
- 不能路径压缩，要 MLE，所以只能按秩合并，每次询问最多 $\mathcal{O}(\log^2 n)$

可持久化并查集

- 直接维护一个主席树表示 x 的祖先。
- 不能路径压缩，要 MLE，所以只能按秩合并，每次询问最多 $\mathcal{O}(\log^2 n)$
- 单次修改时间复杂度 $\mathcal{O}(\log n)$

可持久化并查集

- 直接维护一个主席树表示 x 的祖先。
- 不能路径压缩，要 MLE，所以只能按秩合并，每次询问最多 $\mathcal{O}(\log^2 n)$
- 单次修改时间复杂度 $\mathcal{O}(\log n)$
- 总复杂度 $\mathcal{O}(n \log^2 n)$

目录

- 1 ST 表
- 2 树状数组
- 3 线段树
- 4 珂朵莉树
- 5 可持久化线段树 (主席树)
- 6 我们的红太阳

- Jur_Cai 的宣传[链接](#)。

- 选题完成, AK naive or hard 1600。

作业

- 选题完成, AK naive or hard 1600。
- naive: <https://www.luogu.com.cn/contest/98310>

作业

- 选题完成, AK naive or hard 1600。
- naive: <https://www.luogu.com.cn/contest/98310>
- normal: <https://www.luogu.com.cn/contest/98320>

作业

- 选题完成, AK naive or hard 1600。
- naive: <https://www.luogu.com.cn/contest/98310>
- normal: <https://www.luogu.com.cn/contest/98320>
- hard: <https://www.luogu.com.cn/contest/104611>

GL&HF